

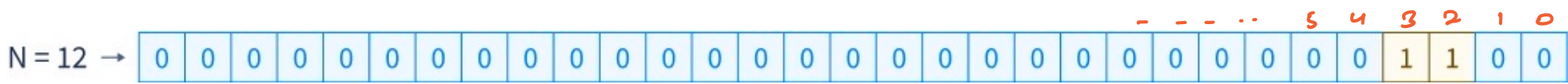
Starting 7:05

Today's Agenda:-

1. Count of Set Bits
2. Finding Good Days
3. Subarrays having OR 1
4. Maximum AND pair.

< Question > : Given an integer N. Count the set-bits in N.

Example :



ans = 2.

Approach:- Iterate on all 32 bits of the integer & check if i^{th} bit is set or not set. Using $A \& (1 \ll i)$.

If set, increment the count. 7:14

Code.

```
int countBits ( int N ) {  
    int ans = 0;  
    for ( i = 0 ; i < 32 ; i++ ) {  
        if ( checkBit ( N , i )  
            == true ) {  
            ans ++;  
        }  
    }  
    return ans;  
}
```

Approach 2: Using a right shift operator.

N = 12

ans = 0



Only check the LSB of the number until it becomes 0.
Every set bit will become LSB once..

Code.

int Count Bits (int N) {

int ans = 0;

while (N > 0) {

if (N & 1 == 1) {

1
3
ans++;

N = N >> 1; // Dividing by 2..

return ans;

TC $\rightarrow O(\log N)$

Problem Statement

Alex has a cat named Boomer. He decides to put his cat to the test for eternity.

He starts on day 1 with one stash of food unit, every next day, the stash doubles.

If Boomer is well behaved during a particular day, only then she receives food worth equal to the stash produced on that day.

Boomer receives a net worth of A units of food. What is the number of days she received the stash?

Day 1 → 1
Day 2 → 2
Day 3 → 4
Day 4 → 8
Day 5 → 16

Day 6 → 32

a & b.

A = 10

A = 15

ar = 4

2.

1010
↓ ↓
2³ 2¹

1 1 1 1
↓ ↓ ↓ ↓
2³ 2² 2¹ 2⁰

Approach:- Count the amount of set bits in stash received in A.

Q:- Given a binary array. Find the total no. of subarrays the bitwise OR of which = 1

which = 1

ex:- { 1, 0, 1 }

Subarrays :- Bitwise OR

2 1 3

2 1, 0 3

2 1, 0, 1 3

2 0 3

2 0, 1 3

2 1 3

1
1
1
0
1
1

ans = 5

Observation:-

1. Bitwise OR of any subarray can either be 0 or 1.

2. For a subarray to give OR as 1, it should have at least one 1.

Approach 1:- Consider any subarray & count 1's in that subarray. If it is ≥ 1 , increment ans. (Prefix Sum)
 $\Rightarrow O(N^2)$.

Approach 2:-

Total Subarrays in an array of size N = Total Subarray having OR as 1 + Total Subarray having OR as 0.
 $\frac{N(N+1)}{2} = x + y$

Ans:-

$$\{ \overset{0}{0}, \overset{1}{0}, \overset{2}{1}, \overset{3}{0}, \overset{4}{0}, \overset{5}{0}, \overset{6}{1}, \overset{7}{1}, \overset{8}{1}, \overset{9}{0}, \overset{10}{1} \}$$

$$\downarrow \qquad \qquad \qquad \downarrow \qquad \qquad \qquad \downarrow$$

$$\frac{2 \times (2+1)}{2} \qquad \qquad \frac{3 \times (3+1)}{2} \qquad \qquad \frac{1 \times (1+1)}{2}$$

$$3 + 6 + 1 = \underline{10}$$

$$\frac{11 \times (12)}{2} = x + 10$$

$$\therefore x = 66 - 10$$

$$= \underline{56}$$

$$\{ \underline{5}, \underline{6} \}$$

$$\underline{8:10}$$

$$\{ \overset{0}{0}, \overset{1}{0}, \overset{2}{1}, \overset{3}{0}, \overset{4}{0}, \overset{5}{0}, \overset{6}{1}, \overset{7}{1}, \overset{8}{1}, \overset{9}{0}, \overset{10}{1} \}$$

$$\text{Count} = 0/1/2/3/4/5/6/7/8/9/10$$

$$\text{ans} = 0 + \frac{2 \times (2+1)}{2} + \frac{3 \times (3+1)}{2}$$

$$+ 0 + 0 + \frac{1 \times (1+1)}{2}$$

$$= 0 + 3 + 6 + 1 = \underline{10}$$

$$\text{Ans} \quad \frac{N(N+1)}{2} - \underline{\text{ans.}} = 66 - 10 = \underline{56}$$

int count = 0; ans = 0;

for (i = 0; i < N; i++) {

if (arr[i] == 0) {
count++;

}

else {

ans += $\frac{\text{count} * (\text{count} + 1)}{2}$;

count = 0;

}

}

ans += $\frac{\text{count} * (\text{count} + 1)}{2}$;

return $\frac{N(N+1)}{2} - \text{ans}$;

T.C $\rightarrow O(N)$

ex:- { 1, 0, 0, 1, 1, 0, 0 }ⁱ

count = 0 1 2 0 0 1 2

ans = 0 + 0 + $\frac{2 * (2+1)}{2}$ + 0

1 / - - -

8:35

Maximum and Pair

< **Question** > : Given N +ve array elements. Find the maximum value of $(arr[i] \& arr[j])$ where $i \neq j$ (indices must be different for 2 number)

arr[] $\rightarrow$ [27 18 20]
 0 1 2

a1 a2 a3 a4 a5
4 13 12 11

27 & 18
27 & 20

18 & 20

27 $\rightarrow$ 1 1 0 1 1
18 $\rightarrow$ 1 0 0 1 0

1 0 0 1 0

&

1 0 0 1 0

18

27 $\rightarrow$ 1 1 0 1 1
20 $\rightarrow$ 1 0 1 0 0

1 0 0 0 0

&

1 0 0 0 0

16

18 $\rightarrow$ 1 0 0 1 0
20 $\rightarrow$ 1 0 1 0 0

1 0 0 0 0

&

1 0 0 0 0

16

Idea 1: Consider every possible pair of the array.

T.C $\rightarrow O(N^2)$

Observation:-

1: Max. value of x can only be / when both the bits are set.

2. Since MSB's contribution is more,
I should try to select 2
no's whose MSB is set;

ex:- { 26, 18, 23, 28, 27, 7, 25 }

	4	3	2	1	0
26 →	1	1	0	1	0
0 18 →	0	X	X	0	X
0 23 →	1	0	X	X	X
0 28 →	1	1	1	0	0
27 →	1	1	0	1	1
0 7 →	0	0	X	X	X
0 25 →	1	1	0	0	X
	5	4	1	2	1

ans = 50

ans :- 1 1 0 1 0

↓
Don't reject any. In future,
it can help.

Ans:

{21, 18, 24, 17, 16}

			4	3	2	1	0
21	→		1	0	1	0	1
0 18	→		1	0	0	1	0
0 24	→		1	1	0	0	0
17	→		1	0	0	0	1
0 16	→		1	0	0	0	0
			5	1	1	1	2

ans:-

1 0 0 0 1

ans = 33

Approach:-

Iterate from MSB to LSB
for each number & Count the set
bits on that index

Count ≥ 2

Count ≤ 1
(0, 1)

1. Set that bit in ans.
2. Reject all no.s that have that bit unset.

1. Iterate to next index.

int ans = 0;

for (i = 31; i >= 0; i--) {

count = 0; // Count set bits at ^{ith} index.

for (j = 0; j < N; j++) {

if (checkBit(A[j], i) == true) {

count++;

if (count >= 2) { // Reject others only if at least 2 elements are true.

ans = ans | (1 << i);

for (j = 0; j < N; j++) {

if (checkBit(A[j], i) == false) {

A[j] = 0;

return ans;

TC $\rightarrow O(N)$

Q:-

2 5, 4, 3, 2, 1 3

		2	1	0
	5 →	1	0	1
	4 →	1	0	0
0	3 →	0	X	0
0	2 →	0	X	0
0	1 →	0	0	X

Ans:-

1 0 0

Q:- How many pairs are possible which can give us maximum AND 2.

Sol:- After computing the previous ans, iterate on the array again to find out elements that are still non-zero. Let that be x.

$$\text{ans} = \frac{x \times (x-1)}{2}$$

$$= \frac{x(x-1)}{2}$$

			4	3	2	1	0	
	21	→	1	0	1	0	1	
0	18	→	1	0	0	1	0	
0	24	→	1	1	0	0	0	
	17	→	1	0	0	0	1	
0	16	→	1	0	0	0	0	
			5	1	1	1	2	

ans:-

1 0 0 0 1

ans = 33

✓ 12	1	1	0	0
✓ 10	1	0	1	0
✓ 9	1	0	0	1
	3	1	1	1

1 0 0 0